

Data Structures & Algorithms

— Modules 1–3 Reviewer

A full revision guide for Modules 1–3 — algorithms & arrays, searching & sorting, and linked lists & stacks. Every search and every sort has an **illustrated step-by-step simulation** run on the exact inputs from the group-activity sheets (GT2B / GT2C / GT2D / GT3A). Compiled from the Canvas module pages and the Lec01–Lec06 decks. **Infix/postfix conversion and expression trees are intentionally excluded.**

M1 · Algorithms & Arrays

M2 · Searching & Sorting

M3 · Linked Lists & Stacks

Illustrated simulations

Print-friendly

M1 Data structures & algorithms

MODULE 1 · ALGORITHMS, DATA STRUCTURES AND ARRAYS · 8 HOURS · LEC01

LEARNING OUTCOMES

- Discuss the different data structures used in computer science and how they apply across science, technology, and business.
- Demonstrate the operations performed on one- and two-dimensional arrays.

A **data structure** is: a representation of the **logical relationship** between data elements; a specialized format to **store and organize data** in memory or on disk; a collection of variables, possibly of different types, connected in various ways. In short — a way of organizing data that considers **both the elements and their relationships**.

An **algorithm** is a **finite set of instructions** specifying a sequence of operations to carry out a task — "a recipe for solving a problem."

PROGRAM = Algorithm + Data Structures

Criteria of a good algorithm

CRITERION	MEANING
Input	Zero or more quantities externally supplied.
Output	At least one quantity is produced.
Definiteness	Each instruction is clear and unambiguous.
Finiteness	All instructions terminate after a finite number of steps.
Effectiveness	Each operation is definite and reliable / basic enough to carry out.

Classification of data structures

	PRIMITIVE	NON-PRIMITIVE
What	Basic structures operated on directly by machine instructions; representation differs across computers.	More sophisticated, <i>derived from</i> primitives; structure groups of homogeneous or heterogeneous items.
Examples	<code>int</code> , float, character/string constants, pointers	List, Stack, Queue, Tree, Graph (also arrays, BST, AVL tree)

Common operations: **Create, Select, Update, Search, Sort, Merge, Destroy/Delete.**

M1 Analysis of algorithms

HOW EFFICIENT IS AN ALGORITHM? · LEC01

Performance is measured by the **computer resources** an algorithm needs (CPU time, memory, I/O) **relative to the input size** — e.g. input size = 500 elements in an array.

TECHNIQUE	DEFINITION
Best case	Function performing the minimum number of steps on input of <code>n</code> elements.
Worst case	Function performing the maximum number of steps on input of size <code>n</code> .
Average case	Function performing an average number of steps on input of <code>n</code> elements.

 **Note:** deep Big-O growth analysis is *not* covered here — that is the *Design of Algorithms (AlgoCom)* course.

M1 Arrays

THE FIRST DATA STRUCTURE · REVIEW ON ARRAYS

An **array** is an **ordered collection of data items of the same type** (homogeneous) under a **single name**, characterised by its **name, dimension, size**, and **element data type**. Also called a **subscripted variable**.

TYPE	DESCRIPTION
One-dimensional	A linear list; components accessed by one index . The index always starts at 0 .
Two-dimensional	An "array of arrays" — two indices; a table / matrix . First index = row , second = column .
Multi-dimensional	More than two dimensions (a 3-D array uses three subscripts). Used in games, animation, CGI.

EIGHT OPERATIONS ON ARRAYS

Reading/accepting · Displaying/retrieving · Searching · Inserting · Sorting · Deleting · Merging · Splitting.

Worked example — accessing elements

GIVEN 1-D array num = { 2, 8, 5, 6, 0 } (indices 0–4)

```
// index      0    1    2    3    4
// num[]      =  2    8    5    6    0
cout << num[1];           // prints 8
addNum  = num[0] + num[4]; // 2 + 0 = 2
prodNum = num[1] * 5;      // 8 * 5 = 40
quoNum  = num[3] / num[0]; // 6 / 2 = 3
```

GIVEN 2-D array num (3×3): row 0 {5,12,9}, row 1 {7,3,34}, row 2 {23,1,0}

```
//          col0  col1  col2
// row0      5    12    9
// row1      7     3   34
// row2     23     1    0
cout << num[1][2];           // prints 34
addNum  = num[0][0] + num[2][1]; // 5 + 1 = 6
prodNum = num[1][2] * 5;       // 34 * 5 = 170
```

❗ **Trap:** indices start at **0**; size $n \rightarrow$ valid indices $0 \dots n-1$. In $\text{num}[i][j]$, i is the row, j the column.

M1 Arrays as matrices

A 2-D ARRAY AS A MATHEMATICAL OBJECT

A **matrix** is a 2-D array of numbers with a fixed number of rows and columns. With **m** rows and **n** columns it is an **m × n** matrix (its **order**).

TYPE	DEFINITION	EXAMPLE
Row matrix	One row.	<code>[0 -8 3]</code> → 1 × 3
Column matrix	One column.	<code>[4 ; 7]</code> → 2 × 1
Vector	One row <i>or</i> one column.	<code>[3 1 4]</code>
Scalar	One row <i>and</i> one column.	<code>[4]</code> → 1 × 1
Zero / null	All elements zero.	<code>[0 0 ; 0 0]</code>
Square	Equal rows and columns.	<code>[1 2 ; 3 4]</code>
Magic square	Distinct integers <code>1...n²</code> ; every row, column and main diagonal has the same sum. For 3×3 that sum is 15 .	<code>[2 7 6 ; 9 5 1 ; 4 3 8]</code> rows/cols/diagonals all = 15
Sparse	More zeros than non-zeros; stored compactly to save memory.	<code>[0 0 3 ; 0 0 0 ; 5 0 0]</code> → keep <code>(0,2,3)</code> , <code>(2,0,5)</code>
Transpose (A^T)	Flip over the main diagonal (swap row/column indices).	$A = [1\ 2\ 3\ ;\ 4\ 5\ 6]$ (2×3) $A^T = [1\ 4\ ;\ 2\ 5\ ;\ 3\ 6]$ (3×2)

 **Trap:** a **scalar** is 1×1 ; a **vector** is one row *or* column; a 3×3 magic square line-sum is **15**.

M2 Searching — terms

MODULE 2 · SEARCHING AND SORTING ALGORITHMS · 6 HOURS · LEC03

- **Search key** — the value that uniquely identifies the requested data.
- **Key field** — contains the key of the record.
- **Information field** — contains the info associated with that key.

You need: the array, its length, and the item sought. Afterwards: report "**success**" + location, or "**not found**" / "**failure**". Search quality is measured by the **number of comparisons** — lower is better.

GIVEN (GT2B)

sorted list = { 12, 25, 37, 48, 61, 74, 89, 90, 95, 101 } keys: 61 and 90

 comparing  found / done  current window  skipped / inactive

M2 Linear (sequential) search — simulated

Checks **every element one at a time in sequence**. Works on *unsorted* data and on linked lists. Not ideal for large lists.

1. start at index 0
2. compare the element with the key
3. if equal → return the index (success)
4. else move to the next index; if past the end → not found

LINEAR SEARCH — KEY 61

cmp 1



$a[0] = 12 \neq 61$

cmp 2



$a[1] = 25 \neq 61$

cmp 3



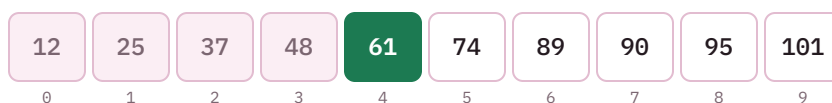
$a[2] = 37 \neq 61$

cmp 4



$a[3] = 48 \neq 61$

cmp 5



$a[4] = 61 = 61 \rightarrow \text{FOUND}$

LINEAR SEARCH – KEY 90

cmp 1



$a[0] = 12 \neq 90$

cmp 2



$a[1] = 25 \neq 90$

cmp 3



$a[2] = 37 \neq 90$

cmp 4



$a[3] = 48 \neq 90$

cmp 5



$a[4] = 61 \neq 90$

cmp 6



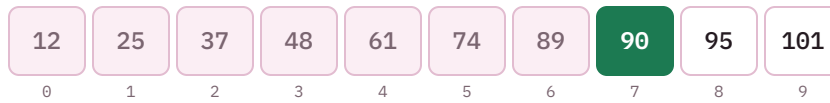
$a[5] = 74 \neq 90$

cmp 7



$a[6] = 89 \neq 90$

cmp 8



$a[7] = 90 = 90 \rightarrow \text{FOUND}$

Key 61 → 5 comparisons (found at index 4). **Key 90** → 8 comparisons (index 7). A missing key costs the full 10 comparisons.



Trap: linear search does *not* need sorted data; it is the only one of the three usable on a linked list.

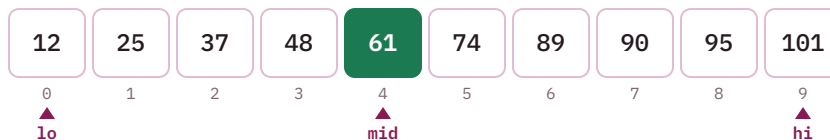
M2 Binary search — simulated

Divide and conquer — halves the search space each step. **Requires a sorted array.**

1. $lo = 0, hi = n-1$
2. $mid = (lo + hi) / 2$ (integer division)
3. if $a[mid] == key \rightarrow$ found
4. if $a[mid] < key \rightarrow lo = mid + 1$ else $hi = mid - 1$
5. repeat while $lo \leq hi$; else not found

BINARY SEARCH — KEY 61

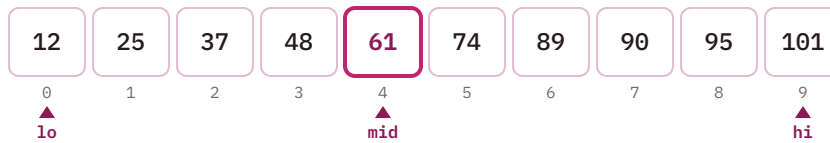
step 1



$lo=0 \ mid=4 \ hi=9 \rightarrow a[4]=61 = 61 \text{ FOUND}$

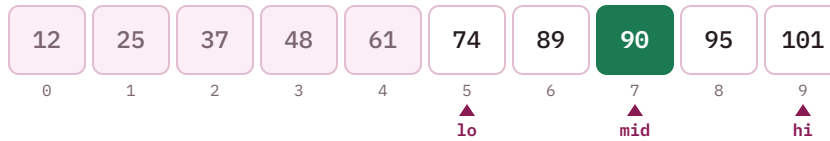
BINARY SEARCH — KEY 90

step 1



lo=0 mid=4 hi=9 → a[4]=61 < 90 lo = 5

step 2



lo=5 mid=7 hi=9 → a[7]=90 = 90 **FOUND**

Key 61 → 1 comparison (mid lands on it). **Key 90 → 2 comparisons**. Far fewer than linear search on the same list.

! Trap: binary search is **impossible on a linked list** (no random access to the middle) and **fails on unsorted data**.

M2 Jump search — simulated

Supplementary — Lec03 lists it under "modern searching algorithms" (with interpolation, exponential, Fibonacci) but gives no detail; GT2B asks you to apply "one modern search of your choice".

Jump search works on a **sorted array**. Fix a block size **step = $\lfloor \sqrt{n} \rfloor$** . **Jump** ahead block by block, checking the element at the end of each block, until that element is **$\geq$ key** (or the end is reached). Then do a **linear scan** backwards from the previous block boundary. One "jump-back" only.

1. `step = floor(sqrt(n)); prev = 0`
2. `while a[min(step, n) - 1] < key: prev = step; step += floor(sqrt(n))`
3. linear-scan indices `prev ... min(step, n)-1`
4. `if a[i] == key → found; else not found`

`n = 10` step = $\lfloor \sqrt{10} \rfloor = 3$ → block boundaries at indices 2, 5, 8

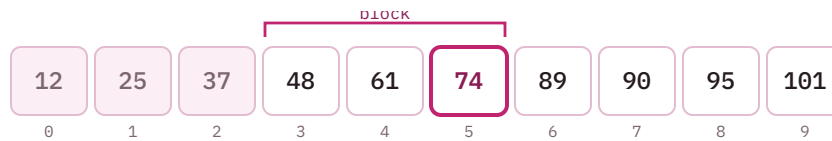
JUMP SEARCH — KEY 61 (STEP = 3)

jump 1



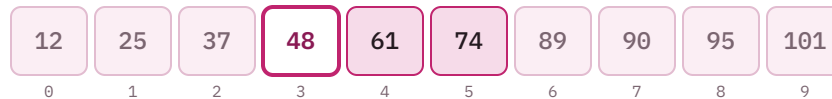
check $a[2] = 37 < 61 \rightarrow$ jump ahead

jump 2



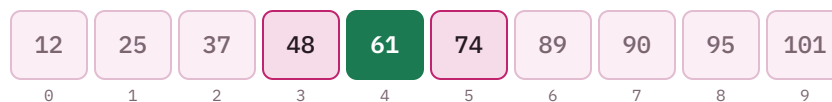
check $a[5] = 74 \geq 61 \rightarrow$ stop; scan block 3-5

scan 1



$a[3] = 48 \neq 61$

scan 2



$a[4] = 61 = 61 \rightarrow$ **FOUND at index 4**

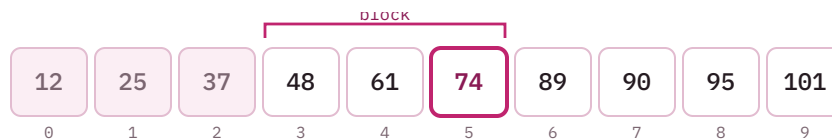
JUMP SEARCH – KEY 90 (STEP = 3)

jump 1



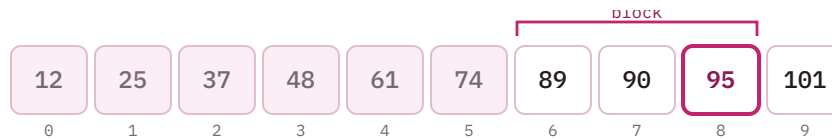
check $a[2] = 37 < 90 \rightarrow$ jump ahead

jump 2



check $a[5] = 74 < 90 \rightarrow$ jump ahead

jump 3



check $a[8] = 95 \geq 90 \rightarrow$ stop; scan block 6-8

scan 1



$a[6] = 89 \neq 90$

scan 2



$a[7] = 90 = 90 \rightarrow$ **FOUND at index 7**

Key 61 $\rightarrow$ 4 comparisons (boundaries $a[2]$, $a[5]$; then scan $a[3]$, $a[4]$). **Key 90** $\rightarrow$ 5 comparisons ($a[2]$, $a[5]$, $a[8]$; then $a[6]$, $a[7]$).

ADVANTAGES

- Fewer comparisons than linear search.
- Only one backward linear scan — simpler than binary search.

DISADVANTAGES

- Slower than binary search.
- Needs a **sorted** array (like binary search).

M2 Sorting fundamentals

LEC04 · BEFORE THE ALGORITHMS

Sorting = arranging data in order (numerical or lexicographical). It matters because searching is far faster on sorted data. Computers repeat two steps: **compare** two items, then **swap** them.

PROPERTY	MEANING	EXAMPLE
In-place	No extra space beyond the array.	Bubble sort
Not-in-place	Needs space $\geq$ the number of elements.	Merge sort
Stable	Keeps the order of equal elements.	Bubble, insertion, merge
Unstable	May reorder equal elements.	Selection, quick, shell

All sorts below produce **ascending** order and run on **arrays**.

■ comparing ■ swapped this step ■ pivot ■ in final position

M2 Bubble sort — simulated

The **simplest** sort. Each pass compares adjacent pairs and swaps the larger one rightward, "bubbling" the largest remaining value to the end.

1. compare two adjacent elements
2. if $\text{left} > \text{right} \rightarrow \text{swap}$
3. move one step right; repeat to the end (one pass)
4. repeat passes until no swaps occur

GIVEN (GT2C) array = { 65, 21, 90, 25 }

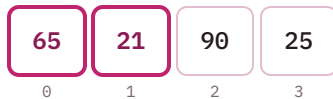
BUBBLE SORT - { 65, 21, 90, 25 }

start



start

p1



pass 1: compare $a[0]=65$ & $a[1]=21 \rightarrow \text{swap}$



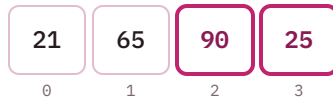
after swap

p1

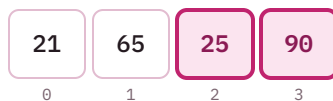


pass 1: compare $a[1]=65$ & $a[2]=90 \rightarrow \text{keep}$

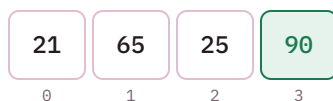
p1



pass 1: compare $a[2]=90$ & $a[3]=25 \rightarrow \text{swap}$

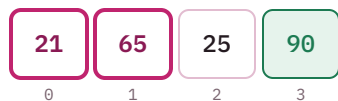


after swap



end of pass 1

p2



pass 2: compare $a[0]=21$ & $a[1]=65 \rightarrow$ keep

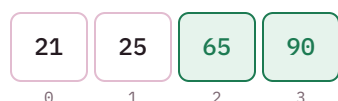
p2



pass 2: compare $a[1]=65$ & $a[2]=25 \rightarrow$ swap

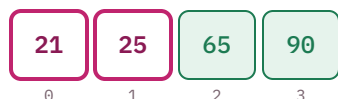


after swap



end of pass 2

p3



pass 3: compare $a[0]=21$ & $a[1]=25 \rightarrow$ keep



end of pass 3 – no swaps, done

done



sorted

ADVANTAGES

- Popular, easy to implement.
- In-place – minimal space.

DISADVANTAGES

- Very slow on large lists.

M2 Selection sort — simulated

Splits the list into a **sorted part (left)** and **unsorted part (right)**. Each pass finds the minimum of the unsorted part and swaps it into the first unsorted slot.

1. find the minimum value in the unsorted part
2. swap it into the first unsorted position
3. repeat for the rest of the list

GIVEN (GT2C) array = { 34, 22, 46, 50, 89, 48, 73, 57, 92 }

SELECTION SORT – { 34, 22, 46, 50, 89, 48, 73, 57, 92 }

start



start

p1



pass 1: min of a[0..8] is 22 (idx 1) → swap with a[0]



a[0] fixed

p2



pass 2: min of a[1..8] is 34 (idx 1) → already in place



a[1] fixed

p3



pass 3: min of a[2..8] is 46 (idx 2) → already in place



a[2] fixed

p4

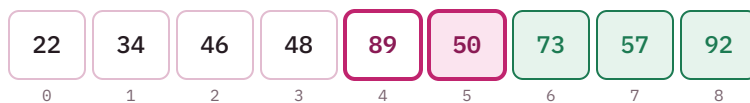


pass 4: min of a[3..8] is 48 (idx 5) → swap with a[3]



a[3] fixed

p5



pass 5: min of a[4..8] is 50 (idx 5) → swap with a[4]



a[4] fixed

p6



pass 6: min of a[5..8] is 57 (idx 7) → swap with a[5]



a[5] fixed

p7



pass 7: min of a[6..8] is 73 (idx 6) → already in place



a[6] fixed

p8



pass 8: min of a[7..8] is 89 (idx 7) → already in place



a[7] fixed

done



sorted

ADVANTAGES

- Good on small lists.
- In-place.

DISADVANTAGES

- Poor on large lists.
- Needs n^2 steps.

M2 Insertion sort — simulated

Builds the sorted list **one entry at a time** — how people sort a hand of cards.

1. the first element is already sorted
2. take the next element (the "key")
3. shift every sorted element greater than the key one place right
4. insert the key; repeat

GIVEN (GT2C) array = { 34, 22, 46, 50, 89, 48, 73, 57, 92 }

INSERTION SORT — { 34, 22, 46, 50, 89, 48, 73, 57, 92 }

start



a[0] is trivially sorted

key 1



take key = a[1] = 22



shift > 22 right, insert 22 at index 0

key 2



take key = a[2] = 46



shift > 46 right, insert 46 at index 2

key 3

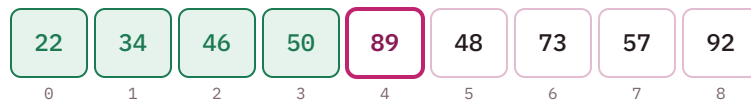


take key = a[3] = 50

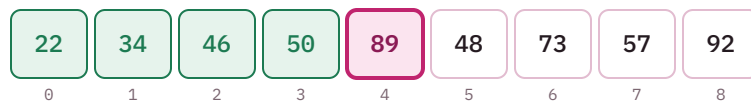


shift > 50 right, insert 50 at index 3

key 4



take key = a[4] = 89

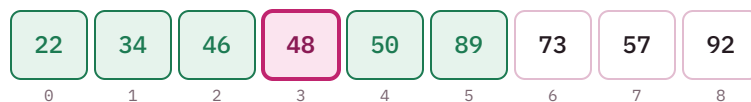


shift > 89 right, insert 89 at index 4

key 5



take key = a[5] = 48



shift > 48 right, insert 48 at index 3

key 6



take key = a[6] = 73



shift > 73 right, insert 73 at index 5

key 7



take key = a[7] = 57



shift > 57 right, insert 57 at index 5

key 8



take key = a[8] = 92



shift > 92 right, insert 92 at index 8

done



sorted

ADVANTAGES

- Simple.
- Great when the input is nearly sorted.
- In-place.

DISADVANTAGES

- n^2 steps — poor on huge lists.

TRY IT

run bubble / selection / insertion on { 47, 99, 25, 104, 10, 71, 88, 33, 62 }
too

M2 Quick sort — Lomuto & Hoare

Also called "**partition-exchange sort**" — one of the most efficient in practice. Pick a **pivot**, partition so smaller elements sit on one side and larger on the other, then recurse on each side. **Divide and conquer**. Pivot strategies: first, last, random, or median element.

GIVEN (GT2D) array = { 47, 99, 25, 104, 10, 71, 88, 33, 62 }

Variant 1 — Lomuto partition (pivot = last element)

One scanning index **i** marks the boundary of " $\leq$ pivot". Walk **j** across the range; whenever $a[j] \leq \text{pivot}$, advance **i** and swap $a[i] \leftrightarrow a[j]$. Finally swap the pivot into $a[i+1]$.

partition(lo, hi):

```
pivot = a[hi]; i = lo - 1
for j = lo .. hi-1:
    if a[j] ≤ pivot: i++; swap a[i], a[j]
swap a[i+1], a[hi]
return i+1                                // pivot's final index
```

QUICK SORT — LOMUTO (PIVOT = LAST) — { 47, 99, 25, 104, 10, 71, 88, 33, 62 }

QS(0,8)



pivot = $a[8] = 62$ partition $a[0..8]$

j=0



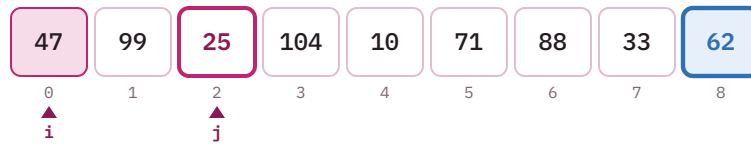
$a[0] = 47 \leq 62 \rightarrow i \rightarrow 0$, swap $a[0] \leftrightarrow a[0]$

j=1



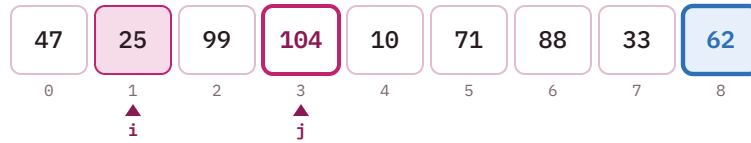
$a[1] = 99 > 62 \rightarrow$ leave

j=2



$a[2] = 25 \leq 62 \rightarrow i \rightarrow 1$, swap $a[1] \leftrightarrow a[2]$

j=3



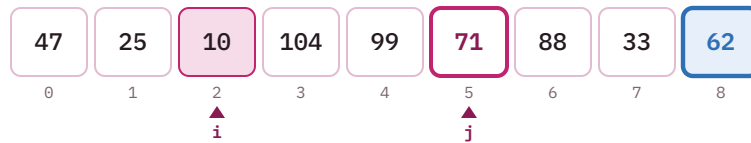
$a[3] = 104 > 62 \rightarrow$ leave

j=4



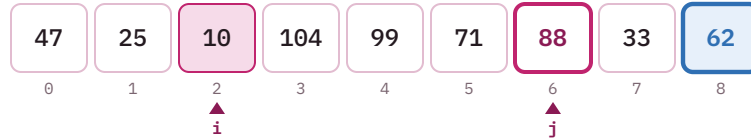
$a[4] = 10 \leq 62 \rightarrow i \rightarrow 2$, swap $a[2] \leftrightarrow a[4]$

j=5



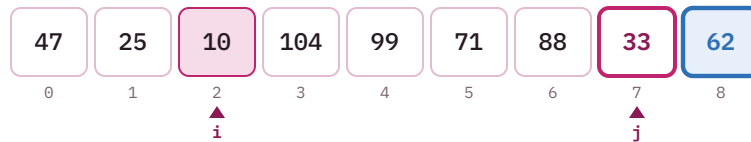
$a[5] = 71 > 62 \rightarrow$ leave

j=6



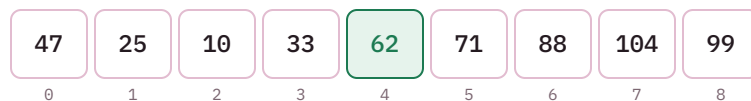
$a[6] = 88 > 62 \rightarrow$ leave

j=7



$a[7] = 33 \leq 62 \rightarrow i \rightarrow 3$, swap $a[3] \leftrightarrow a[7]$

place



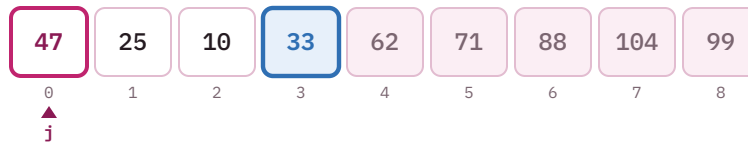
swap pivot into $a[4] \rightarrow$ **62 fixed at index 4**

QS(0,3)



pivot = $a[3] = 33$ partition $a[0..3]$

j=0



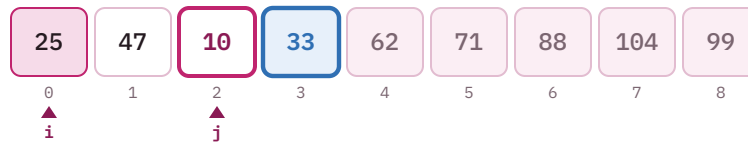
$a[0] = 47 > 33 \rightarrow \text{leave}$

j=1



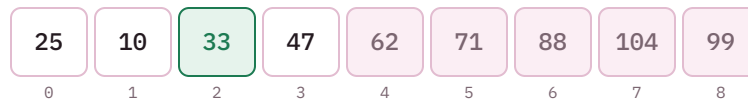
$a[1] = 25 \leq 33 \rightarrow i \rightarrow 0, \text{ swap } a[0] \leftrightarrow a[1]$

j=2



$a[2] = 10 \leq 33 \rightarrow i \rightarrow 1, \text{ swap } a[1] \leftrightarrow a[2]$

place



swap pivot into $a[2] \rightarrow 33$ fixed at index 2

QS(0,1)



pivot = $a[1] = 10$ partition $a[0..1]$

j=0



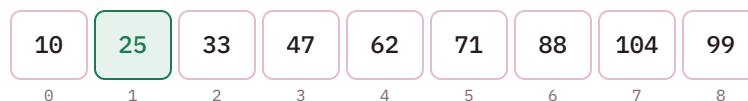
$a[0] = 25 > 10 \rightarrow \text{leave}$

place



swap pivot into $a[0] \rightarrow 10$ fixed at index 0

QS(1,1)



single element – sorted

QS(3,3)



single element – sorted

QS(5,8)



pivot = a[8] = 99 partition a[5..8]

j=5



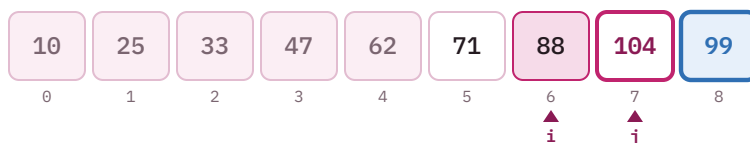
a[5] = 71 ≤ 99 → i→5, swap a[5]↔a[5]

j=6



a[6] = 88 ≤ 99 → i→6, swap a[6]↔a[6]

j=7



a[7] = 104 > 99 → leave

place



swap pivot into a[7] → 99 fixed at index 7

QS(5,6)



pivot = a[6] = 88 partition a[5..6]

j=5



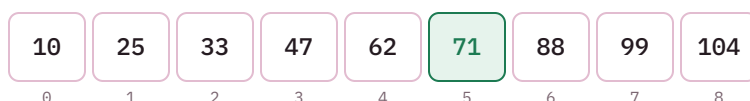
a[5] = 71 ≤ 88 → i→5, swap a[5]↔a[5]

place



swap pivot into a[6] → 88 fixed at index 6

QS(5,5)



single element – sorted

10	25	33	47	62	71	88	99	104
0	1	2	3	4	5	6	7	8

done

10	25	33	47	62	71	88	99	104
0	1	2	3	4	5	6	7	8

```
recursion: QS(0,8)→pivot 62 · QS(0,3)→pivot 33 · QS(0,1)→pivot 10 · QS(5,8)→pivot 99 ·
QS(5,6)→pivot 88
```

Variant 2 — Hoare partition (pivot = first element, two pointers)

Two indices move inward: **i** from the left stops at an element $\geq \text{pivot}$; **j** from the right stops at an element $\leq \text{pivot}$; swap them; continue until they cross. Returns a **split point** (not the pivot's final index) and does **fewer swaps** than Lomuto.

```
partition(lo, hi):
```

```
pivot = a[lo];  i = lo - 1;  j = hi + 1
```

loop:

```
do i++ while a[i] < pivot
```

```
do j-- while a[j] > pivot
```

```
if i ≥ j: return j
```

```
swap a[i], a[j]
```

```
quicksort(lo, hi): if lo < hi: p = partition(lo,hi); quicksort(lo,p); quicksort(p+1,hi)
```

QUICK SORT - HOARE (PIVOT = FIRST, TWO POINTERS) - SAME INPUT

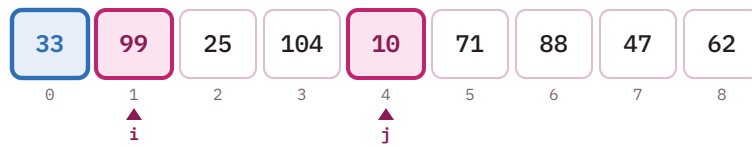
47	99	25	104	10	71	88	33	62
0	1	2	3	4	5	6	7	8

```
pivot = a[0] = 47    partition a[0..8]
```

Diagram illustrating an array of 9 elements: 47, 99, 25, 104, 10, 71, 88, 33, 62. The first element (47) is highlighted with a blue border, and the last element (33) is highlighted with a pink border. Below the array, indices 0 through 8 are shown. A purple arrow labeled i points to index 0, and a pink arrow labeled i points to index 7.

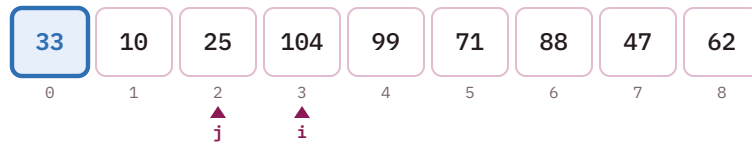
$a[0]=47 \geq \text{pivot}, a[7]=33 \leq \text{pivot} \rightarrow \text{swap}$

swap



$a[1]=99 \geq \text{pivot}$, $a[4]=10 \leq \text{pivot} \rightarrow \text{swap}$

split



$i=3 \geq j=2 \rightarrow \text{split here; recurse } a[0..2] \text{ and } a[3..8]$

QS(0,2)



$\text{pivot} = a[0] = 33$ partition $a[0..2]$

swap



$a[0]=33 \geq \text{pivot}$, $a[2]=25 \leq \text{pivot} \rightarrow \text{swap}$

split



$i=2 \geq j=1 \rightarrow \text{split here; recurse } a[0..1] \text{ and } a[2..2]$

QS(0,1)



$\text{pivot} = a[0] = 25$ partition $a[0..1]$

swap



$a[0]=25 \geq \text{pivot}$, $a[1]=10 \leq \text{pivot} \rightarrow \text{swap}$

split



$i=1 \geq j=0 \rightarrow \text{split here; recurse } a[0..0] \text{ and } a[1..1]$

QS(3,8)



pivot = $a[3] = 104$ partition $a[3..8]$

swap



$a[3]=104 \geq \text{pivot}$, $a[8]=62 \leq \text{pivot} \rightarrow \text{swap}$

split



$i=8 \geq j=7 \rightarrow \text{split here; recurse } a[3..7] \text{ and } a[8..8]$

QS(3,7)



pivot = $a[3] = 62$ partition $a[3..7]$

swap



$a[3]=62 \geq \text{pivot}$, $a[7]=47 \leq \text{pivot} \rightarrow \text{swap}$

split



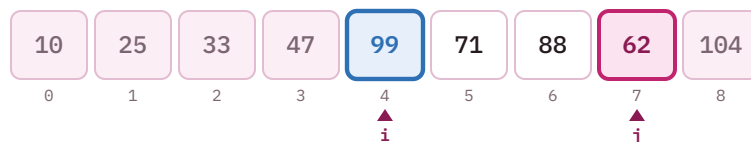
$i=4 \geq j=3 \rightarrow \text{split here; recurse } a[3..3] \text{ and } a[4..7]$

QS(4,7)



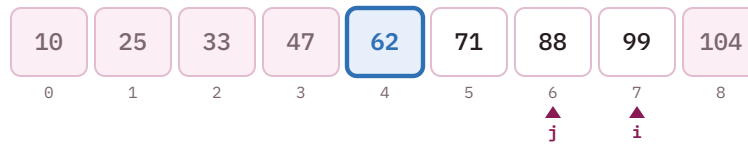
pivot = $a[4] = 99$ partition $a[4..7]$

swap



$a[4]=99 \geq \text{pivot}$, $a[7]=62 \leq \text{pivot} \rightarrow \text{swap}$

split



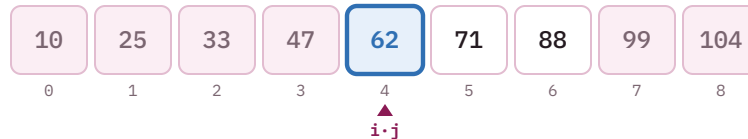
$i=7 \geq j=6 \rightarrow$ split here; recurse $a[4..6]$ and $a[7..7]$

QS(4,6)



pivot = $a[4] = 62$ partition $a[4..6]$

split



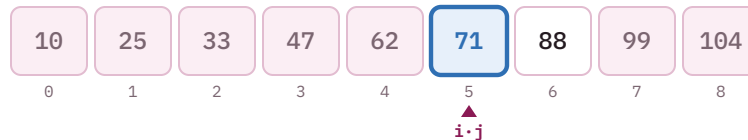
$i=4 \geq j=4 \rightarrow$ split here; recurse $a[4..4]$ and $a[5..6]$

QS(5,6)



pivot = $a[5] = 71$ partition $a[5..6]$

split



$i=5 \geq j=5 \rightarrow$ split here; recurse $a[5..5]$ and $a[6..6]$

done



sorted

Both variants finish at **10 25 33 47 62 71 88 99 104**.

ADVANTAGES

- Fastest general sort in practice; handles huge lists.
- Sorts in place.

DISADVANTAGES

- Worst case $\approx$ average of the simple sorts.
- On **already-sorted** input it degrades — bubble sort beats it.
- For integers, radix sort is faster.



Trap: Lomuto pivot = **last** element, one index, returns the pivot's final position. Hoare pivot = **first** (or middle), two indices moving inward, returns a split point.

M2 Merge sort — simulated

Split into two halves of $n/2$, sort each **recursively**, then **merge** the two sorted runs. **Divide and conquer**, worst case $O(n \log n)$, **not-in-place**.

1. a one-element list is already sorted → return
2. divide recursively into halves until single elements
3. merge pairs of runs back together in sorted order

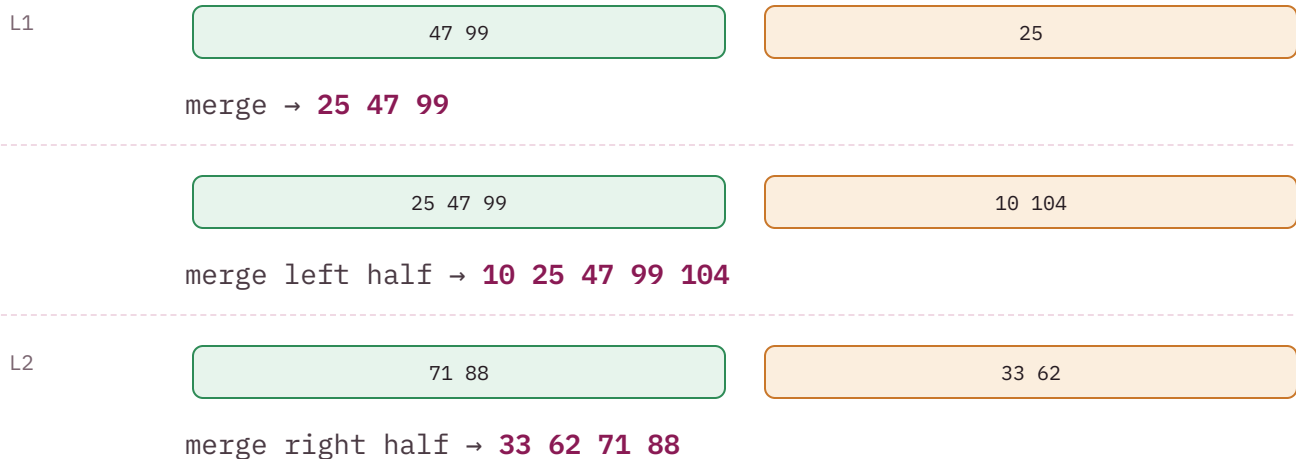
GIVEN (GT2D) array = { 47, 99, 25, 104, 10, 71, 88, 33, 62 }

DIVIDE



MERGE (BOTTOM-UP)

MERGE SORT — COMBINE RUNS



root

10 25 47 99 104

33 62 71 88

final merge → **10 25 33 47 62 71 88 99 104**

ADVANTAGES

- Guaranteed $O(n \log n)$; stable; great for very large data.

DISADVANTAGES

- Not-in-place — needs extra memory equal to the input.

M2 Shell sort — simulated

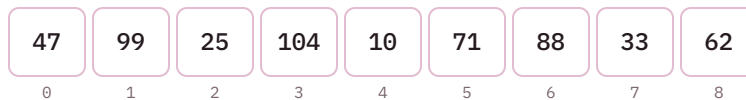
An improved **insertion sort** that first sorts elements a fixed distance apart — the **gap** — then shrinks it. **gap** = $\text{floor}(n / 2)$, halved until 1.

GIVEN (GT2D)

array = { 47, 99, 25, 104, 10, 71, 88, 33, 62 }, $n = 9 \rightarrow$ gaps 4, 2, 1

SHELL SORT — GAPS 4 → 2 → 1

start



start

gap 4



gap 4: consider $a[4] = 10$ against $a[0] = 47$



shift & place 10 at index 0

gap 4



gap 4: consider $a[5] = 71$ against $a[1] = 99$



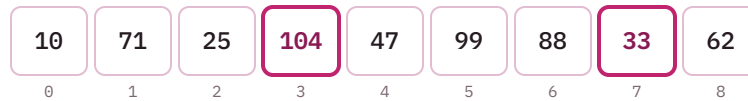
shift & place 71 at index 1

gap 4



gap 4: consider $a[6] = 88$ against $a[2] = 25$

gap 4



gap 4: consider $a[7] = 33$ against $a[3] = 104$



shift & place 33 at index 3

gap 4



gap 4: consider $a[8] = 62$ against $a[4] = 47$



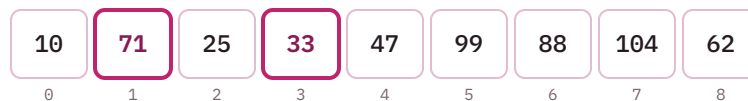
after gap 4: 10 71 25 33 47 99 88 104 62

gap 2



gap 2: consider $a[2] = 25$ against $a[0] = 10$

gap 2

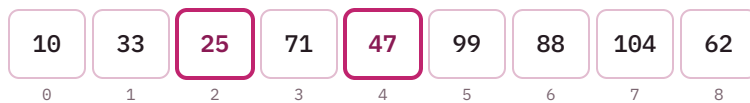


gap 2: consider $a[3] = 33$ against $a[1] = 71$



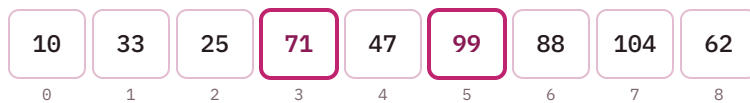
shift & place 33 at index 1

gap 2



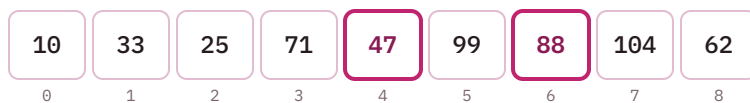
gap 2: consider $a[4] = 47$ against $a[2] = 25$

gap 2



gap 2: consider $a[5] = 99$ against $a[3] = 71$

gap 2



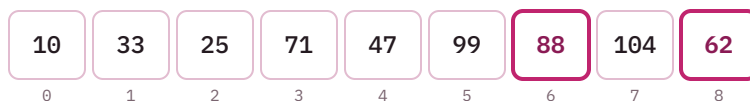
gap 2: consider $a[6] = 88$ against $a[4] = 47$

gap 2



gap 2: consider $a[7] = 104$ against $a[5] = 99$

gap 2



gap 2: consider $a[8] = 62$ against $a[6] = 88$



shift & place 62 at index 6



after gap 2: 10 33 25 71 47 99 62 104 88

gap 1

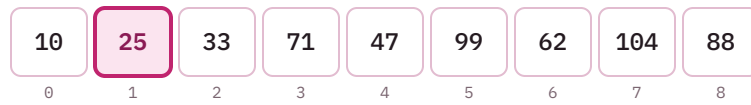


gap 1: consider $a[1] = 33$ against $a[0] = 10$

gap 1



gap 1: consider $a[2] = 25$ against $a[1] = 33$



shift & place 25 at index 1

gap 1



gap 1: consider $a[3] = 71$ against $a[2] = 33$

gap 1



gap 1: consider $a[4] = 47$ against $a[3] = 71$



shift & place 47 at index 3

gap 1



gap 1: consider $a[5] = 99$ against $a[4] = 71$

gap 1



gap 1: consider $a[6] = 62$ against $a[5] = 99$



shift & place 62 at index 4

gap 1

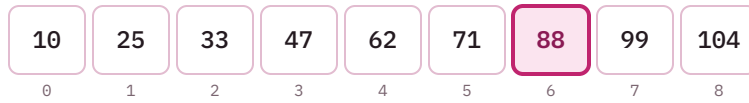


gap 1: consider $a[7] = 104$ against $a[6] = 99$

gap 1



gap 1: consider $a[8] = 88$ against $a[7] = 104$



shift & place 88 at index 6



after gap 1: 10 25 33 47 62 71 88 99 104

done



sorted

ADVANTAGES

- Very efficient for medium arrays.
- Better than insertion; ~5× faster than bubble.

DISADVANTAGES

- Complex; not as fast as merge/quick; slows on large arrays.

M2 Sorting comparison table

ALGORITHM	IDEA	IN-PLACE	STABLE	NOTES
Bubble	swap adjacent out-of-order pairs	Yes	Yes	simplest; slow on large lists
Selection	repeatedly pick the min	Yes	No	n^2 ; small lists only
Insertion	build sorted part one key at a time	Yes	Yes	great on nearly-sorted input
Quick — Lomuto	pivot = last; one index; partition + recurse	Yes	No	simple to code; more swaps
Quick — Hoare	pivot = first; two indices inward	Yes	No	fewer swaps; returns a split point
Shell	gapped insertion, gap = $\text{floor}(n/2)$	Yes	No	medium arrays; ~5× faster than bubble
Merge	split to halves, sort, merge	No	Yes	$O(n \log n)$ worst; extra memory

M3 Linked lists — SLL, DLL, CLL

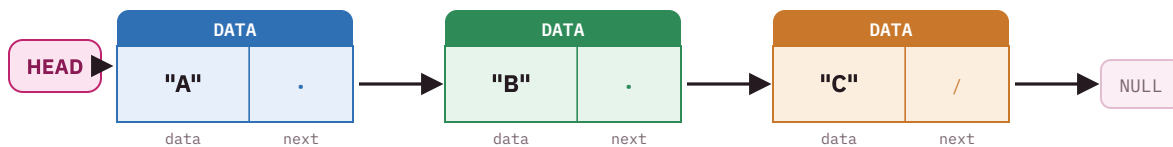
MODULE 3 · LINKED LISTS AND STACKS · 9 HOURS · LEC06

A **linked list** is a **dynamic, linear** data structure whose elements are **not contiguous in memory** — they are joined by **pointers**. Node count is not fixed and can **grow / shrink on demand**. Second most-used structure after the array.

NODE

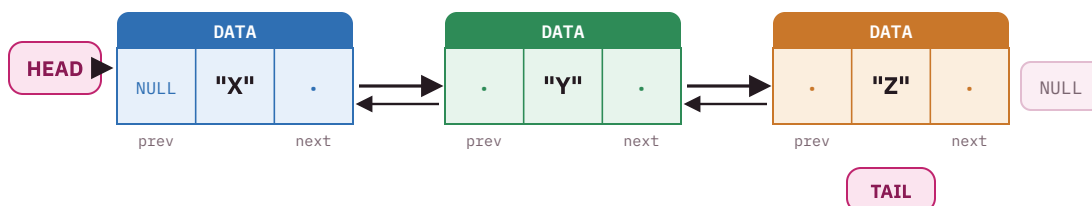
- **Data** — the value stored.
- **Link / Next** — address of the next node (the **pointer field**).
- **Head** — holds the address of the first node. Last node's **next** = **NULL**.

1 · Singly linked list (SLL) — uni-directional



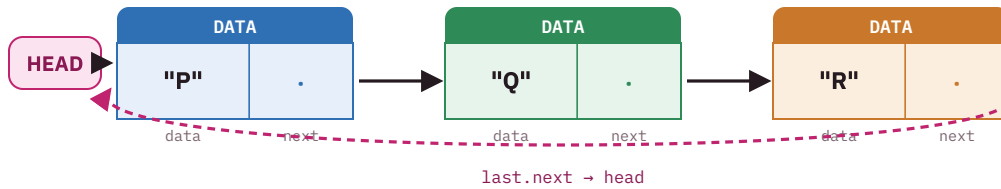
One pointer per node (**next**); traversal is **forward only**; the last node points to **NULL**; the next node is the **successor**.

2 · Doubly linked list (DLL) — bi-directional



Two pointers per node: **next** (→ **successor**) and **prev** (→ **predecessor**). The list keeps **HEAD** and **TAIL**; the first node's **prev** and the last node's **next** are **NULL**. Easier to implement and debug than a singly list.

3 · Circular linked list (CLL) — last links back to head



The last node's `next` points to the **first node** instead of NULL (in a circular *doubly* list the first node's `prev` also points to the last). **There is no NULL terminator** — a traversal must stop when it returns to HEAD.

Linked list vs. array

	ARRAY	LINKED LIST
Memory	one contiguous block reserved up front	nodes anywhere; allocated / freed on demand
Access	direct / random by index ($O(1)$)	sequential only — from head, follow links
Insert / delete	expensive — shift elements	cheap — relink pointers
Size	fixed at allocation	grows and shrinks freely

DRAWBACKS OF ARRAYS

- Fixed size, rarely fully used.
- Insertion / deletion are expensive (shifting).

DRAWBACKS OF LINKED LISTS

- No random access.
- **Binary search is impossible.**
- Extra pointer memory per node.
- Worse cache locality.

APPLICATIONS

CS: stacks & queues; graph adjacency lists; dynamic memory (free-block lists); directory of names; long-integer arithmetic; polynomials; sparse matrices. **Real world:** image viewer (prev/next), browser history (back/forward), music player (previous/next song).

M3 Linked list operations — simulated

Build a singly list (insert at head)

Node = [data | next]; start with HEAD = NULL . Each new node's next becomes the old head, then HEAD points to the new node.

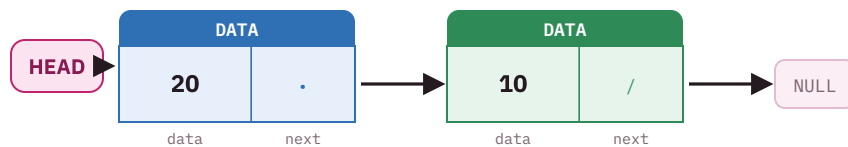
BUILD A SINGLY LIST BY INSERTING AT THE HEAD

ins 10



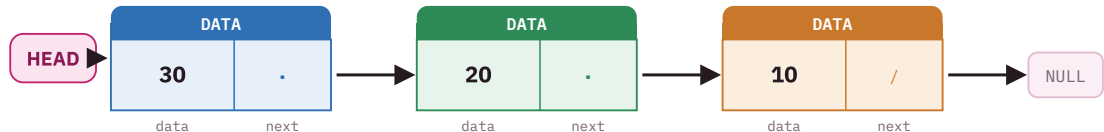
insert 10 → HEAD → [10 | /]

ins 20



insert 20: new node.next = old head, HEAD = new node

ins 30



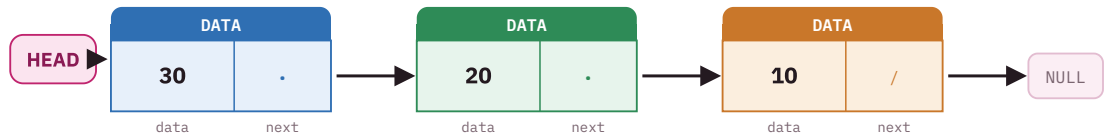
insert 30 → list is 30 → 20 → 10 → NULL (length 3)

Find the length — pseudocode + simulation

- 1) $n = 0$
- 2) $\text{current} = \text{head}$
- 3) while $\text{current} \neq \text{NULL}$: $\text{current} = \text{current} \rightarrow \text{next}$; $n++$
- 4) return n

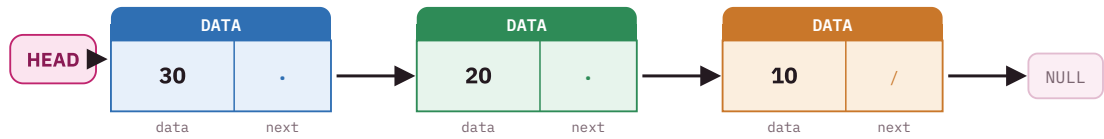
TRAVERSE TO COUNT NODES

init



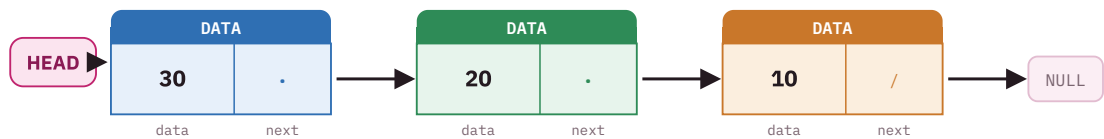
$n = 0$, $\text{current} = \text{HEAD}$ (node 30)

iter 1



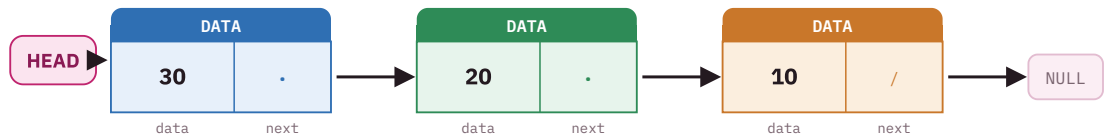
$\text{current} \neq \text{NULL} \rightarrow \text{current} = \text{node } 20$, $n = 1$

iter 2



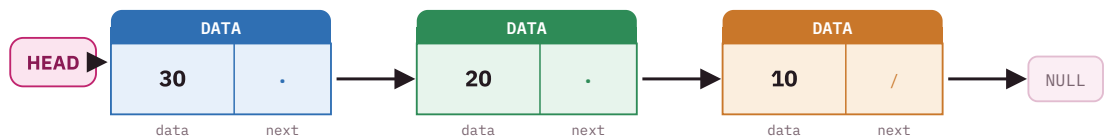
$\text{current} \neq \text{NULL} \rightarrow \text{current} = \text{node } 10$, $n = 2$

iter 3



$\text{current} \neq \text{NULL} \rightarrow \text{current} = \text{NULL}$, $n = 3$

stop



$\text{current} == \text{NULL} \rightarrow \text{return } n = 3$

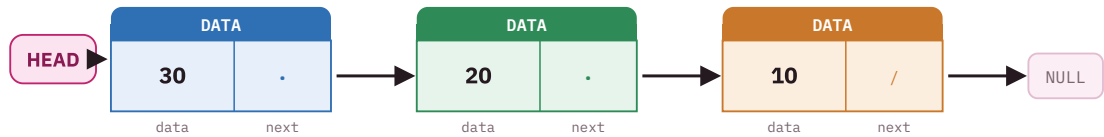
Retrieving the i th node and storing a new value at position i also require a full traversal to reach the node first.

Insert a node in the middle ($1 < i < n$)

Insert **25** at position 2 of $30 \rightarrow 20 \rightarrow 10$. Three places a node can go: start ($i=1$), end ($i > \text{length}$), or middle.

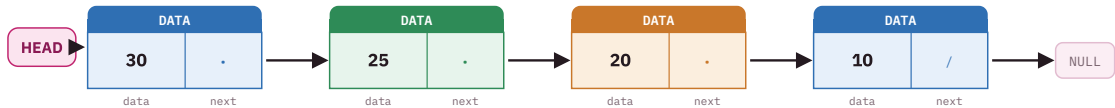
INSERT 25 AT POSITION 2 OF $30 \rightarrow 20 \rightarrow 10$

find



traverse: pre = node 30, cur = node 20

splice

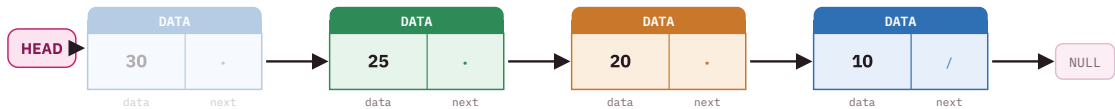


newNode.next = cur (20); pre.next = newNode (25) → 30 → 25 → 20 → 10

Delete the head (i = 1)

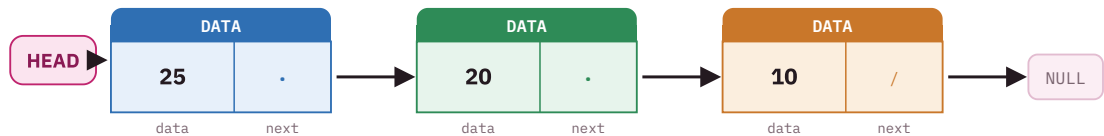
DELETE THE HEAD NODE (I = 1)

unlink



temp = HEAD (30); HEAD = HEAD->next (25)

done



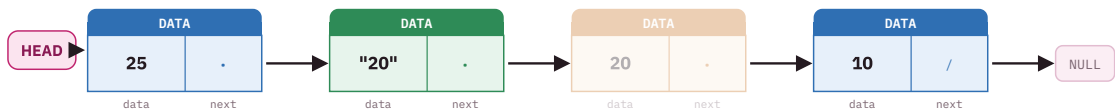
free temp → 25 → 20 → 10 → NULL

Delete a node NOT at the head ($1 < i \leq n$)

Delete node 20: link the preceding node past it.

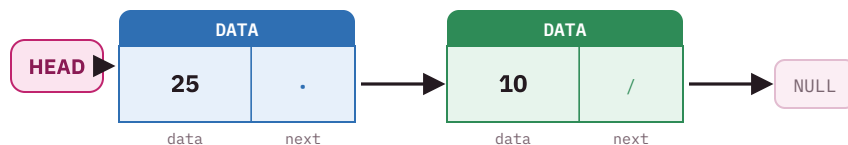
DELETE NODE 20 (NOT THE HEAD)

find



locate pre = node 25, cur = node 20

relink

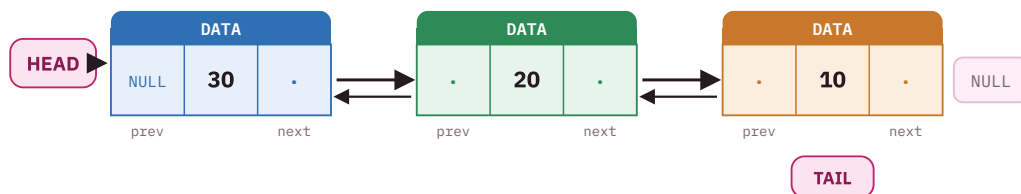


`pre.next = cur.next → free cur → 25 → 10 → NULL`

Doubly list — delete the head, then a middle node

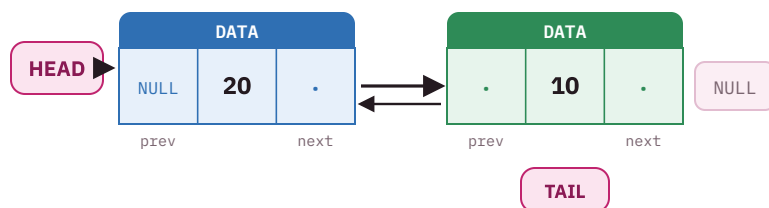
DOUBLY LIST — DELETE HEAD, THEN A MIDDLE NODE

start



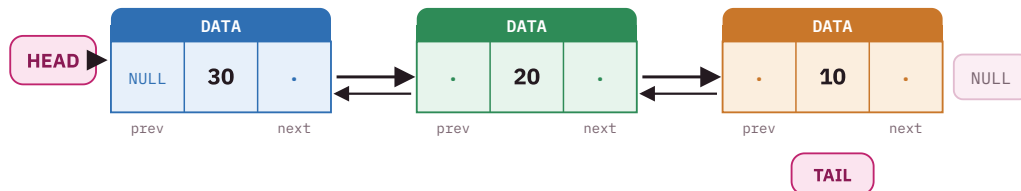
start: 30 ↔ 20 ↔ 10

del head



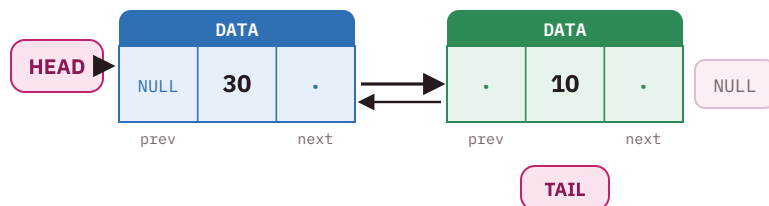
delete head: `HEAD = HEAD->next (20); 20.prev = NULL`

reset



rebuild 30 ↔ 20 ↔ 10, then delete middle (20)

del mid



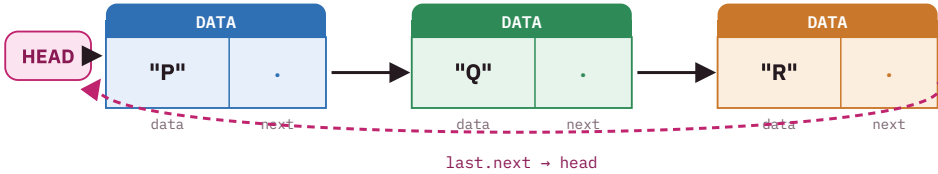
pred 30, succ 10: `30.next = 10; 10.prev = 30 → 30 ↔ 10`

Circular list — traversal & insert at end

Traversal stops when `current->next == head` (no NULL). Inserting at the end makes the new node's `next` point back to `head`.

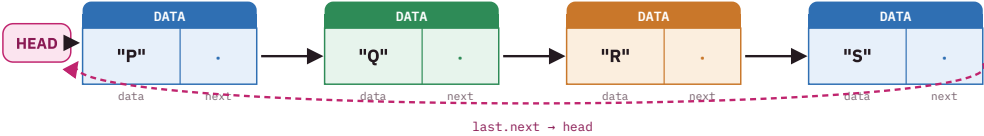
CIRCULAR LIST – TRAVERSAL & INSERT AT END

traverse



traverse from HEAD: visit P, Q, R; at R, R.next == head → **stop**

insert



insert S at end: R.next = S; S.next = head (P)

GT3A worked simulation

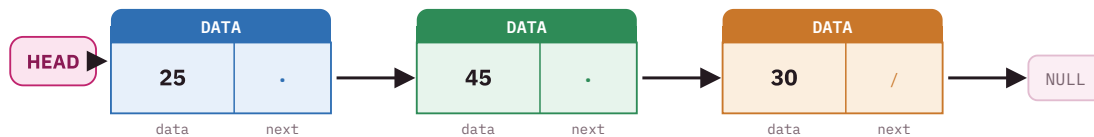
GT3A

HEAD = A. A: 25 → E | E: 45 → C | C: 30 → NULL. Tasks: insert 15 as a new head; remove 45.

LOCATION	DATA	NEXT	AFTER INSERT 15 (@B)	AFTER REMOVE 45
A	25	E	E	C
B	—	—	15 → A (new HEAD)	15 → A
C	30	NULL	NULL	NULL
E	45	C	C	freed

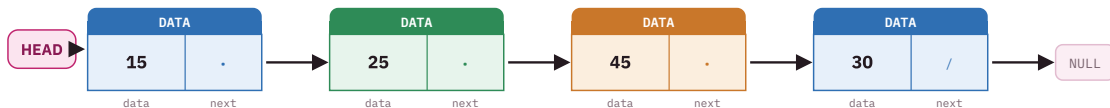
GT3A – INSERT 15 AS NEW HEAD, THEN REMOVE 45

start



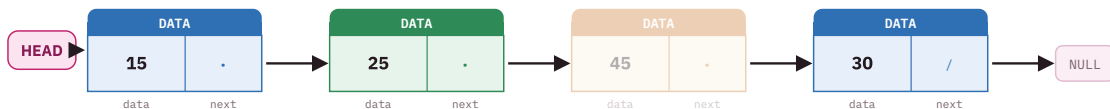
start (HEAD = A): 25 → 45 → 30 → NULL

ins 15



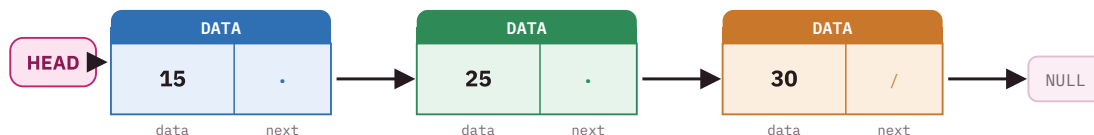
insert 15 at slot B: B.next = A; HEAD = B → 15 → 25 → 45 → 30

find 45



remove 45: predecessor A; A.next = 45.next = C

done



free node 45 → 15 → 25 → 30 → NULL

M3 Stacks

A LIMITED-ACCESS DATA STRUCTURE · IMPLEMENTATION OF STACKS

A **stack** is an ordered list where **all operations are restricted to one end** — the **top**. It follows **LIFO** (Last In, First Out). It is a **recursive** structure: a stack is empty, or a **top** plus **the rest, which is itself a stack**.

Analogy: a stack of cafeteria trays.

OPERATION	EFFECT	TOP AFTER
PUSH	add an item on top	TOP++
POP	remove the top item	TOP--
PEEK	read the top value, no removal	unchanged

TOP holds the *index* of the top element: **-1** / **NULL** when empty, **0** for one element, incremented on push, decremented on pop. A stack is stored in a **1-D array** or a **singly linked list** (this course uses the array).
Overflow = push onto a full stack; **underflow** = pop from an empty one.

Worked trace — push / pop / peek

GIVEN PUSH 5, PUSH 8, PUSH 2, POP, PUSH 9, PEEK, POP, POP

STACK – PUSH 5, PUSH 8, PUSH 2, POP, PUSH 9, PEEK, POP, POP

init

empty

TOP = -1

empty stack, TOP = -1

PUSH 5

5

← TOP (idx 0)

TOP = 0

PUSH 5, TOP = 0

PUSH 8

8

5

← TOP (idx 1)

TOP = 1

PUSH 8, TOP = 1

PUSH 2

2

8

5

← TOP (idx 2)

TOP = 2

PUSH 2, TOP = 2

POP

8

5

← TOP (idx 1)

TOP = 1 returns 2

POP → returns **2**, TOP = 1

PUSH 9



TOP = 2

PUSH 9, TOP = 2

PEEK



TOP = 2 returns 9

PEEK → returns **9**, TOP = 2

POP



TOP = 1 returns 8

POP → returns **8**, TOP = 1

POP



TOP = 0 returns 5

POP → returns **5**, TOP = 0

```
// array implementation – Implementation of Stacks.pdf
int stack[100], n = 100, top = -1;
void push(int val){ if(top >= n-1) cout<<"Stack Overflow";
                  else{ top++; stack[top]=val; } }
void pop(){ if(top <= -1) cout<<"Stack Underflow";
            else{ cout<<"Popped: "<<stack[top]; top--; } }
```

M3 Stack applications — simulated

Function calls · evaluation of expressions · reversing a word · undo in a text editor · backtracking (maze) · language processing (parameter/local-variable space, matching-brace check, recursion support).

Function calls

PROC1 calls FUNC1, which calls FUNC2. Each call pushes a frame; each return pops it.

CALL STACK — PROC1 → FUNC1 → FUNC2



TOP = 0

PROC1 starts



TOP = 1

PROC1 calls FUNC1 → push



TOP = 2

FUNC1 calls FUNC2 → push



TOP = 1

FUNC2 returns → pop; control → FUNC1 (stmt b)



TOP = 0

FUNC1 returns → pop; control → PROC1 (stmt a)



TOP = -1

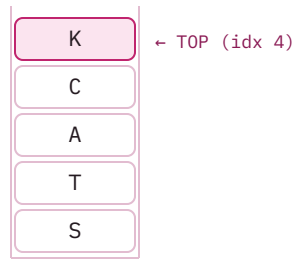
PROC1 returns → pop

Reversing a word

GIVEN word = "STACK"

REVERSE "STACK" WITH A STACK → "KCATS"

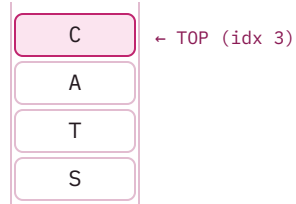
push all



TOP = 4

PUSH S, T, A, C, K

pop



TOP = 3

pop → output so far: **K**

pop



TOP = 2

pop → output so far: **KC**

pop



TOP = 1

pop → output so far: **KCA**

pop



TOP = 0

pop → output so far: **KCAT**

pop



TOP = -1

pop → output so far: **KCATS**

❗ **Out of scope:** the module continues into *evaluation of expressions* — infix vs. postfix notation, the infix→postfix conversion algorithm, and expression trees (GT3D). Those are **not covered in this reviewer** by request.

★ Quick-recall cheat sheet

PROGRAM

Algorithm + Data Structures

ALGO CRITERIA

**Input · Output ·
Definiteness ·
Finiteness ·
Effectiveness**

ALGO ANALYSIS

**Best · Worst ·
Average case**

ARRAY INDEX

starts at 0
size $n \rightarrow 0 \dots n-1$

3×3 MAGIC SQUARE

every line = 15

LINEAR SEARCH

any order; $O(n)$
works on linked lists

BINARY SEARCH

**needs sorted;
halves range**
impossible on a linked list

JUMP SEARCH

**needs sorted;
block = $\text{floor}(\sqrt{n})$**
jump, then one linear scan back

IN-PLACE

**bubble yes · merge
no**

STABLE

**bubble, insertion,
merge**

QUICK – LOMUTO

**pivot = last; 1
index**

QUICK – HOARE

**pivot = first; 2
indices inward**

SHELL GAP

$\text{floor}(n / 2)$, halved

MERGE SORT

**D&C; $O(n \log n)$
worst; not-in-place**

SINGLY LIST

**forward only; last
next = NULL**

DOUBLY LIST

**prev + next; HEAD
+ TAIL**

CIRCULAR LIST

**last next $\rightarrow$ head;
no NULL**

DELETE HEAD

**HEAD = HEAD-
>next**

STACK

LIFO; top only

STACK OPS

**PUSH top++ · POP
top-- · PEEK same**

TOP EMPTY

-1 / NULL

STACK STORAGE

**1-D array or singly
linked list**

★ Key terms glossary

TERM	MEANING
Data structure	Organizing data while keeping the logical relationship between elements.
Algorithm	A finite set of unambiguous instructions that solves a task.
Primitive / non-primitive	machine-level type (int, float, char, pointer) / built from primitives (list, stack, queue, tree, graph).
Best / worst / average case	analysis by min / max / mean steps for input size n.
Array / subscript	ordered homogeneous collection under one name / an element's index (from 0).
Matrix / transpose / sparse	2-D number array $m \times n$ / flipped over the diagonal / mostly zeros, stored compactly.
Search key	value that uniquely identifies the requested data.
Linear search	check each element in order until the key is found.
Binary search	divide-and-conquer search on sorted data; halves the range each step.
Jump search	sorted-array search; jump by blocks of $\text{floor}(\sqrt{n})$, then linear-scan one block.
Block / step	the jump distance in jump search.
In-place / stable sort	uses no extra space / keeps the order of equal elements.
Pivot	the element quicksort partitions around.
Lomuto partition	pivot = last element; single scanning index; returns the pivot's final index.
Hoare partition	pivot = first (or middle); two indices move inward and swap; returns a split point.
Gap / interval	distance between compared elements in shell sort; $\text{floor}(n/2)$.
Divide and conquer	split, solve sub-problems, combine (quick, merge, binary search).
Node / head / tail	a list element (data + link[s]) / pointer to first node / pointer to last node.
Successor / predecessor	the next / previous node.
Singly / doubly / circular list	forward-only / bidirectional / last node links back to the head.

TERM	MEANING
Stack / LIFO	list with access only at the top / Last In, First Out.
PUSH / POP / PEEK / TOP	add to top / remove top / read top / index of the top element (−1 when empty).
Overflow / underflow	push onto a full stack / pop from an empty stack.

★ Common exam traps

1. **Program = Algorithm + Data Structures** — verbatim.
 2. Array indices start at **0**; `a[i][j]` → `i` is the row.
 3. 5 algorithm criteria — the forgotten three are **Definiteness, Finiteness, Effectiveness**.
 4. A 3×3 **magic square** line-sum is **15**; a **scalar** is 1×1; a **vector** is one row or column.
 5. **Binary and jump search need sorted data**; both are **impossible on a linked list**. Linear search needs neither.
 6. **Jump search block** = `floor(√n)`; only **one** backward linear scan.
 7. **Bubble = in-place; Merge = not-in-place**. Selection, quick, shell are **unstable**.
 8. Quick sort **Lomuto**: pivot = **last**, one index. **Hoare**: pivot = **first**, two indices inward, fewer swaps.
 9. Quick sort degrades on **already-sorted** input — bubble sort beats it there.
 10. **Shell gap** = `floor(n / 2)`, halved until 1; built on **insertion sort**.
 11. Merge sort worst case is `O(n log n)` and it needs **extra memory equal to the input**.
 12. Selection & insertion sort take `n2` steps — small lists only.
 13. **Singly list**: forward only, last `next` = **NULL**, next node = **successor**.
 14. **Doubly list** adds `prev` (the **predecessor**) and a **TAIL**.
 15. **Circular list** has **no NULL** — traversal stops when it reaches **HEAD** again.
 16. **Deleting the head** is just `HEAD = HEAD->next`; deleting elsewhere needs the **preceding node (i-1)**.
 17. **Stack = LIFO**, only **PUSH / POP / PEEK**; `TOP = -1 / NULL` when empty; **peek does not change TOP**.
 18. A stack is stored in a **1-D array or a singly linked list**.
 19. Machine problems in this course are coded in **Java**; the focus is **analysis & simulation**, not coding.
-

simulation uses the exact inputs from GT2B, GT2C, GT2D and GT3A. Jump search is standard supplementary content (Lec03 names it only). Infix/postfix conversion and expression trees are deliberately omitted. For personal revision.